



INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA (ISEL)

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA (DEI)

MEIC & MEIM

MESTRADO EM ENGENHARIA INFORMÁTICA E DE COMPUTADORES

MESTRADO EM ENGENHARIA INFORMÁTICA E MULTIMÉDIA

COMPUTAÇÃO DE DADOS EM LARGA ESCALA

Lab 1: TPC-DS *on a Single Machine*

Miguel Alcobia (50746)

Fábio Pestana (50756)

Docente

Professor António Teófilo

Setembro, 2026

Máquina de teste

Tabela 1: Caracterização da máquina utilizada no Lab 1.

Item	Valor
Modelo do computador	ASUS TUF Dash F15 FX517ZE
CPU e número de cores	Intel Core i7-12650H (10 cores)
RAM	16 GB DDR5
Disco (tipo e espaço livre)	SSD NVMe 512 GB (65 GB livres)
Sistema operativo	Windows 11 x64 / Ubuntu 26.04.1 LTS
Versão do DuckDB	1.5.5

Pipeline

Tabela 2: Comparações nos dados SF1 e SF10

Etapa	Tempo	Tamanho	Observações
SF1: Dados brutos	26,18 s	1,2 GB	4 paralelos
SF10: Dados brutos	5 min 39,11 s	12 GB	4 paralelos
SF1: Parquet	3,9 s	272 MB	25 tabelas, com OK
SF10: Parquet	105,5 s	2,7 GB	25 tabelas, com OK

Confirmou-se 2 880 404 linhas em `store_sales` no SF1 e 28 800 991 no SF10 (valor esperado no apêndice A). Em ambas as escalas a conversão para Parquet reduziu o tamanho cerca de $4,4\times$, graças à codificação por colunas e à compressão zstd.

Data model

Uma *fact table* regista eventos do negócio (uma linha por transação), com colunas maioritariamente chaves estrangeiras para as *dimension table* e algumas medidas numéricas. No TPC-DS existem sete: `store_sales`, `store_returns`, `catalog_sales`, `catalog_returns`, `web_sales`, `web_returns` e `inventory`, representando vendas, devoluções e inventário nos três canais (loja, catálogo, *web*). Uma *dimension table* descreve o contexto de um facto (atributos descritivos como nomes, categorias, datas) e tem uma chave primária substituta (`*_sk`) referenciada pelas *fact table*.

Na parte 4, `store_sales` (2 880 404 linhas no SF1) é um exemplo de *fact table*. Por sua vez a `ship_mode` é uma *dimension table* com apenas 20 linhas que descreve como as mercadorias são enviadas.

Query fixes

A primeira correção resolve a ausência das variáveis `_BEGIN/_END` no ficheiro de dialeto `netezza.tpl`, usadas pelo `dsqgen` para delimitar cada *query*. A sua falta no kit v4.0.0 causa o erro “*Substitution ‘_END’ is used before being initialized*”, o *script* acrescenta as duas definições, associando-as a comentários SQL identificativos.

A segunda correção (terceira em `fix_templates.sh`) renomeia o *alias* `at` da *query* 90 para `at1`, já que `at` é palavra reservada em DuckDB e causa erro de sintaxe.

Experiment A

Tabela 3: Experiência A (SF1 e SF10)

Escala	Run 1	Run 2	Run 3	Soma das medianas
SF1	5,12 s	4,93 s	5,01 s	4,96 s
SF10	46,68 s	46,92 s	49,78 s	47,39 s

No SF1 o *run* 1 é o mais lento (5,12 s), cerca de 3% mais lento que a média dos *runs* seguintes, o que é consistente com o efeito do *cache*: no primeiro *run*, o sistema operativo ainda não tem os ficheiros Parquet em *cache*, e o DuckDB ainda não construiu as suas próprias estruturas internas (planos, estatísticas). Nos *runs* seguintes, esses dados já estão em memória, pelo que as operações de leitura no disco são substituídas por acessos à *cache* do sistema operativo em memória, sendo este mais rápido.

No SF10 o padrão é diferente: os dois primeiros *runs* são semelhantes mas o terceiro é o mais lento, ao contrário do esperado. Devido ao tamanho do *dataset* a variação entre *runs* passa a refletir sobretudo ruído do sistema (como por exemplo portátil quente, ou variações na gestão de memória interna do DuckDB ao lidar com estruturas maiores), e não um efeito do *cache*.

Em ambos os casos, o *benchmark* reporta a mediana de várias execuções para assim evitar este tipo de ruído: uma medição ficar afetada num *run* lento (ou rápido) devido a fatores externos e não representativos do desempenho normal do sistema.

Experiment B

Tabela 4: Experiência B

<i>Threads t</i>	$T(t)$ (s)	$S(t)$	p	q14 (s)	q23 (s)	q67 (s)
SF1: 1	12,95	1,00	—	0,825	1,168	0,730
SF1: 2	7,56	1,71	0,83	0,459	0,695	0,432
SF1: 4	4,97	2,61	0,82	0,287	0,356	0,309
SF10: 1	137,79	1,00	—	8,766	15,416	9,632
SF10: 2	76,98	1,79	0,88	4,635	9,163	5,970
SF10: 4	50,26	2,74	0,85	2,887	4,700	5,555

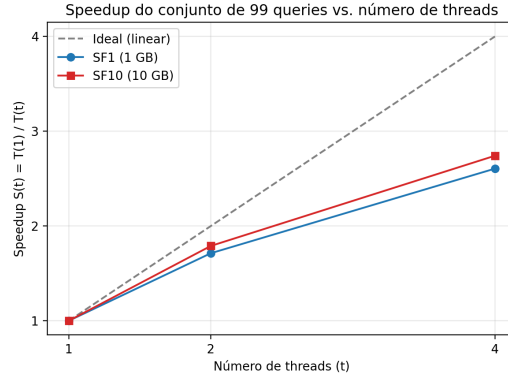


Figura 1: $S(t)$ em função do número de *threads* (SF1 e SF10)

O *speedup* não é linear em nenhuma das escalas ($2,6\times$ e $2,7\times$ a 4 *threads*, em vez de $4\times$), refletindo a lei de Amdahl: há sempre uma fração do trabalho que não paraleliza. O p estimado é ligeiramente mais alto no SF10 do que no SF1, porque no SF1 muitas *queries* demoram apenas milissegundos e o *overhead* fixo (*parsing*, planejamento, arranque do processo Python) pesa proporcionalmente mais. No SF10 o tempo total é dominado pelo *scan*/junção/agregação, que o DuckDB paraleliza melhor. As *queries* pesadas escalam melhor que o conjunto todo, sobretudo a q23 ($3,28\times$ a 4 *threads* nos dois *datasets*), por serem dominadas por operações que paralelizam bem, ao contrário das *queries* pequenas onde domina o *overhead* fixo.

From 1GB to 10GB

Comparando o tempo total do conjunto das 99 *queries* a 4 *threads*, $50,26/4,97 \approx 10,1\times$, próximo do fator de escala mas não exatamente $10\times$. Isto deve-se ao

overhead fixo das *queries* pesarem menos no SF10, o que empurra a razão acima de 10×, por outro lado, nem todas as tabelas escalam por 10, a *inventory* cresce 11,3× enquanto as *dimension table* pequenas (*date_dim*, *ship_mode*, etc.) não crescem.

Optional parts

Part 4. Criou-se o ficheiro `09_top_states.sql`, que junta *customer* a *customer_address* através de *c_current_addr_sk*, agrupa por *ca_state* e devolve os cinco estados com mais clientes. SF1: TX (7 799), GA (4 955), VA (4 330), KY (3 849) e NC (3 303). SF10: TX (39 761), GA (24 415), VA (21 318), KY (18 279) e MO (16 826).

Experiment C Correram-se seis *queries* diretamente sobre os ficheiros `.dat` e depois sobre Parquet. A soma das medianas foi de 2,81 s no formato texto e de 0,39 s em Parquet (7 vezes mais rápido). O ganho vem do *layout* baseado em colunas (só se leem as colunas necessárias), da compressão (cerca de 3 vezes menos bytes), dos tipos binários nativos (sem *parsing* de texto) e das estatísticas mín/máx por *row group*, que permitem saltar dados. O efeito é maior na *query* 7, por ser a que faz um *scan* de uma tabela grande e mais beneficia de ler apenas algumas colunas.

Experiment D A *query* 67 demora 0,248 s no total. A maioria do custo está no operador `HASH_GROUP_BY` (2,03 s acumulados), que agrega 539 305 linhas de *store_sales* em 517 493 grupos através de um `ROLLUP`, seguido do operador `WINDOW` que calcula `RANK()` (0,79 s). Os *scans* e as junções são baratos: a *store_sales* é lida em 0,37 s com *dynamic filters* aplicados diretamente no *scan* Parquet, e as *dimension table* são pequenas.

Prediction for Lab 2

Partição e *skew*, uso de um assistente para gerar/criticar esquemas de particionamento, confirmando as nossas respostas com os *papers*.